

Optimización de Políticas de Juego en Entornos Dinámicos (Tetris) mediante Aprendizaje por Refuerzo y Extracción de Características

Equipo del curso CC421 – Inteligencia Artificial

29 de junio de 2026

Resumen

El juego Tetris constituye un problema clásico de Inteligencia Artificial debido a la naturaleza secuencial de las decisiones, el elevado número de estados posibles y la incertidumbre introducida por la aparición aleatoria de nuevas piezas. Estas características convierten al problema en un entorno adecuado para el estudio y evaluación de técnicas de Aprendizaje por Refuerzo y métodos de optimización de políticas.

El presente trabajo propone el desarrollo de un agente inteligente capaz de aprender estrategias de juego para Tetris mediante una representación basada en características geométricas del tablero y una aproximación lineal de la función de valor. El entorno se modela formalmente como un Proceso de Decisión de Markov, mientras que la calidad de cada estado se estima a partir de un conjunto de atributos que incluyen la altura agregada, el número de huecos, la rugosidad de la superficie, los pozos y las líneas eliminadas.

Con el propósito de optimizar los parámetros de la función de valor se estudiaron dos enfoques complementarios. El primero emplea Aprendizaje por Diferencias Temporales TD(0), permitiendo la actualización incremental de los pesos a partir de la experiencia del agente. El segundo utiliza el Método de Entropía Cruzada, el cual optimiza directamente los parámetros de la política mediante un procedimiento de búsqueda estocástica basado en poblaciones.

Para el caso del agente TD(0), se realizó un barrido exhaustivo de veintisiete configuraciones de hiperparámetros, obteniéndose una política capaz de eliminar en promedio 431.21 líneas antes de alcanzar el estado terminal. Los resultados experimentales demuestran que la combinación de modelado formal mediante MDP, representación basada en características y métodos de optimización de funciones de valor constituye una estrategia efectiva para resolver problemas de toma de decisiones secuenciales en entornos dinámicos de gran complejidad.

Palabras clave: Aprendizaje por Refuerzo, Tetris, Proceso de Decisión de Markov, Temporal Difference Learning, Cross-Entropy Method, Aproximación de Funciones.

1. Introducción

La Inteligencia Artificial ha experimentado un crecimiento significativo en las últimas décadas, impulsando el desarrollo de sistemas capaces de aprender, razonar y tomar decisiones en entornos de elevada complejidad. Dentro de este contexto, el Aprendizaje por Refuerzo constituye una de las áreas de investigación más relevantes, debido a su capacidad para construir agentes que mejoran su comportamiento a partir de la interacción directa con el entorno.

Los videojuegos representan un escenario particularmente adecuado para la evaluación de algoritmos de aprendizaje, ya que presentan problemas de decisión secuencial, objetivos clara-

mente definidos y dinámicas complejas. Entre ellos, el juego Tetris ha sido ampliamente utilizado como un banco de pruebas para la investigación en Inteligencia Artificial debido a la gran dimensionalidad de su espacio de estados y al carácter estocástico de la secuencia de piezas.

Desde una perspectiva computacional, la obtención de una política óptima para Tetris resulta un problema extremadamente complejo. El número de configuraciones posibles del tablero crece de manera exponencial, haciendo inviable la utilización de métodos de búsqueda exhaustiva o técnicas de enumeración completa del espacio de estados.

Ante esta dificultad, se han propuesto diversas estrategias basadas en funciones heurísticas, métodos evolutivos y algoritmos de Aprendizaje por Refuerzo. Particularmente, las aproximaciones basadas en funciones de valor y representaciones compactas del tablero han demostrado ser capaces de producir políticas competitivas con un costo computacional relativamente bajo.

El presente trabajo aborda el problema de Tetris mediante la construcción de un agente inteligente basado en una aproximación lineal de la función de valor. El entorno se modela formalmente como un Proceso de Decisión de Markov y los estados se representan mediante un conjunto de características geométricas extraídas del tablero de juego.

Asimismo, se estudian dos estrategias de optimización de los parámetros de la función de valor. La primera emplea el algoritmo de Diferencias Temporales TD(0), permitiendo un aprendizaje incremental basado en la experiencia del agente. La segunda utiliza el Método de Entropía Cruzada, un procedimiento de optimización estocástica que busca directamente configuraciones de parámetros de alto rendimiento.

El objetivo principal del trabajo consiste en analizar la capacidad de estos métodos para aprender políticas de juego eficientes y estudiar la influencia de los hiperparámetros sobre el desempeño final del agente.

2. Estado del Arte

El problema de Tetris ha sido objeto de investigación durante más de tres décadas y constituye uno de los entornos clásicos para el estudio de algoritmos de Inteligencia Artificial y Aprendizaje por Refuerzo. Su interés radica en la elevada dimensionalidad del espacio de estados, la incertidumbre introducida por la generación aleatoria de piezas y la necesidad de tomar decisiones secuenciales de largo plazo (Russell and Norvig, 2021; Sutton and Barto, 2018).

Los primeros trabajos se basaron en agentes heurísticos capaces de evaluar la calidad de un tablero mediante un conjunto de características geométricas. Entre ellos destaca el controlador propuesto por Dellacherie, el cual utiliza medidas como la altura de las columnas, el número de huecos y la irregularidad de la superficie para determinar la mejor acción posible (Dellacherie, 2003).

Posteriormente, diversos investigadores exploraron técnicas de optimización de políticas. Szita y Lőrincz introdujeron el Método de Entropía Cruzada para el aprendizaje automático de agentes de Tetris, demostrando que los métodos de optimización estocástica pueden superar el rendimiento de numerosos controladores heurísticos (Szita and Lőrincz, 2006).

Con el desarrollo del Aprendizaje por Refuerzo, surgieron nuevas aproximaciones basadas en la estimación de funciones de valor. Thiery y Scherrer propusieron controladores fundamentados en la evaluación de afterstates y en la aproximación de funciones, obteniendo políticas de alto desempeño mediante representaciones compactas del tablero (Thiery and Scherrer, 2009).

Posteriormente, Gabillon, Ghavamzadeh y Scherrer demostraron que las técnicas de Programación Dinámica Aproximada pueden alcanzar desempeños competitivos en Tetris, evidenciando la importancia de la representación del estado y de las funciones de valor aproximadas (Gabillon et al., 2013).

Más recientemente, el auge del Aprendizaje Profundo ha permitido la construcción de agentes basados en Deep Reinforcement Learning, utilizando redes neuronales profundas para aproximar funciones de valor y políticas de decisión. No obstante, estos métodos suelen requerir grandes

volúmenes de datos, recursos computacionales considerables y tiempos de entrenamiento significativamente mayores (Goodfellow et al., 2016; Sutton and Barto, 2018).

En contraste, las aproximaciones lineales continúan siendo una alternativa atractiva debido a su bajo costo computacional, facilidad de interpretación y capacidad para proporcionar resultados competitivos en problemas de mediana complejidad (Gabillon et al., 2013).

Por estas razones, el presente trabajo adopta un enfoque basado en representación mediante características y aproximación lineal de funciones, incorporando tanto Aprendizaje por Diferencias Temporales como el Método de Entropía Cruzada para el aprendizaje y optimización de las políticas de juego.

3. Modelo: formulación como MDP

Modelamos Tetris como un proceso de decisión de Markov (MDP) episódico $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. El carácter *episódico* proviene de que toda partida termina en un estado absorbente (*top-out*) tras un número finito de pasos, lo que da un retorno bien definido sin necesidad de descuento.

3.1. Elementos

- **Estado** $s = (b, p)$: tablero $b \in \{0, 1\}^{H \times W}$ ($H = 20$, $W = 10$) y pieza actual $p \in \{1, \dots, 7\}$. Un estado alcanzable nunca contiene una fila completa, pues se elimina al fijar.
- **Acciones** $\mathcal{A}(s) = \{(r, c)\}$: una *colocación* es una rotación r y una columna de anclaje c factibles para la pieza actual (Sección 3.3).
- **Transición** $P(s' | s, a)$: factoriza en una parte determinista y una aleatoria (Sección 3.4).
- **Recompensa** $R(s, a) = L(s, a)$: líneas eliminadas por la colocación (Sección 3.5).
- **Descuento** $\gamma = 1$ (Sección 3.6).

3.2. Terminología

Un *tetrominó* es una pieza de cuatro celdas; empleamos los siete estándar (I, O, T, S, Z, J, L), cada uno con entre una y cuatro *rotaciones*. El *hard-drop* deja caer la pieza en línea recta hasta que apoya y la *fija* (*lock*); el *soft-drop* es una caída controlada que todavía permite maniobrar. Al fijarse, toda fila completa se elimina (*line clear*) y las superiores descienden. El *afterstate* σ es el tablero resultante tras fijar y limpiar, antes de revelar la pieza siguiente (*spawn*). La partida termina en *top-out* cuando la nueva pieza no admite ninguna colocación. Llamamos *hueco* a una celda vacía con alguna celda ocupada por encima, *pozo* a una columna estrecha y profunda, y *altura* de la columna j a $h_j = H - \min\{i : b_{i,j} = 1\}$.

3.3. Acciones por colocación: finitud y cota

Cada acción es un par (r, c) . El número de rotaciones distintas de cualquier tetrominó es a lo sumo 4. Fijada una rotación de ancho w_r con $1 \leq w_r \leq 4$, las columnas de anclaje válidas son $W - w_r + 1 \leq W$. En consecuencia,

$$|\mathcal{A}(s)| \leq \sum_r (W - w_r + 1) \leq 4W,$$

y $\mathcal{A}(s)$ es finito por ser un producto de conjuntos finitos (rotaciones y columnas). El conteo real es menor, ya que se descartan las colocaciones que desbordan la pila. La formulación por colocación reduce el control a *una decisión por pieza*.

3.4. Transición y mapa de colocación

El estado siguiente tiene dos componentes, $s' = (\sigma', p')$ (el tablero y la pieza siguiente), así que la transición es una distribución *conjunta*. La descomponemos con la regla de la cadena:

$$P((\sigma', p') \mid s, a) = \mathbb{P}(\sigma' \mid s, a) \cdot \mathbb{P}(p' \mid \sigma', s, a). \quad (1)$$

El *mapa de colocación* $\sigma = f(s, a)$ es determinista: dado (s, a) , el hard-drop más la limpieza de líneas producen un *único* tablero. Una distribución con toda su masa en un solo valor es una delta, que escribimos con la función indicadora

$$\mathbf{1}\{C\} = \begin{cases} 1 & \text{si la condición } C \text{ es verdadera,} \\ 0 & \text{en caso contrario,} \end{cases} \quad (2)$$

de modo que el primer factor de (1) es

$$\mathbb{P}(\sigma' \mid s, a) = \mathbf{1}\{\sigma' = f(s, a)\}. \quad (3)$$

El segundo factor es la pieza siguiente, que se sortea de forma independiente del tablero, de la acción y de la historia (es exógena), y la modelamos uniforme sobre las siete:

$$\mathbb{P}(p' \mid \sigma', s, a) = \mathbb{P}(p') = \frac{1}{7}, \quad p' \sim \text{Unif}\{1, \dots, 7\}. \quad (4)$$

Sustituyendo (3) y (4) en (1) se obtiene la transición:

$$P((\sigma', p') \mid s, a) = \mathbf{1}\{\sigma' = f(s, a)\} \cdot \frac{1}{7}. \quad (5)$$

Es decir, desde (s, a) solo son alcanzables los siete sucesores $(\sigma, 1), \dots, (\sigma, 7)$ y equiprobables; para cualquier $\sigma' \neq f(s, a)$ la probabilidad es nula. Elegir el generador independiente (i.i.d.) preserva la propiedad de Markov sin ampliar el estado: un aleatorizador por bolsas (*7-bag*) obligaría a recordar las piezas ya emitidas. Esta factorización es la que habilita trabajar con el afterstate σ (Sección 5).

3.5. Recompensa y su rango

$L(s, a)$ es el número de filas completas eliminadas por la colocación. Como un tetrominó ocupa cuatro celdas repartidas en a lo sumo cuatro filas, y una fila solo puede completarse si la pieza aporta alguna celda a ella, se eliminan a lo sumo cuatro filas. El mínimo 0 (ninguna fila completada) y el máximo 4 (una *I* vertical que cierra cuatro filas a la vez, un *Tetris*) son alcanzables, de modo que

$$L(s, a) \in \{0, 1, 2, 3, 4\}.$$

3.6. Decisiones de modelado

Recompensa. Tomamos $R = L$ porque el objetivo del problema es maximizar el total de líneas de la partida: es una señal acotada y directamente alineada con la métrica de evaluación.

Descuento $\gamma = 1$. Toda línea vale igual sin importar cuándo se elimina; un $\gamma < 1$ introduciría un sesgo temporal (preferir limpiar antes) ajeno al objetivo, que además penalizaría acumular para un *Tetris*. Al ser la tarea episódica y terminar casi seguramente, la suma no descontada es finita; y el optimizador (CEM) maximiza $J(w) = \mathbb{E}[\sum_t R_t]$ de forma directa, sin *bootstrapping*, por lo que $\gamma = 1$ equivale a “sumar las líneas”. En evaluación se acota la longitud del episodio para mantener J finito.

Abstracción del tiempo. El modelo prescinde de la tasa de refresco (*frame rate*) y del tiempo real: un paso del MDP equivale a *una pieza colocada*, y la dinámica intra-pieza (gravedad, soft-drop, desplazamientos laterales) se colapsa en la elección de la posición final más el hard-drop. Se asume, pues, una *decisión inmediata*: lo relevante para la calidad de la política es *dónde* reposa la pieza, no la trayectoria ni el cronometraje para llevarla allí. Como implicación directa, **no se modelan las maniobras que requieren rotar o desplazar la pieza durante la caída** (*tucks*, *T-spins*): $\mathcal{A}(s)$ enumera únicamente las caídas rectas alcanzables por hard-drop. Tampoco se modelan la aceleración de la gravedad por nivel ni el puntaje por soft-drop. El bucle de frames solo sería relevante para una demostración en vivo, donde la colocación elegida se traduciría a una secuencia de entradas de control.

4. Representacion mediante Caracteristicas

El espacio de estados de Tetris es muy grande. Por ello, no resulta practico almacenar un valor independiente para cada configuracion posible del tablero. En su lugar, se utiliza una representacion compacta basada en caracteristicas numericas del tablero.

Sea el vector de caracteristicas:

$$\phi(s) = (f_1(s), f_2(s), f_3(s), f_4(s), f_5(s), f_6(s)). \quad (6)$$

Cada componente resume una propiedad relevante del tablero despues de colocar una pieza.

4.1. Altura agregada

Sea h_j la altura de la columna j . La altura agregada se define como la suma de las alturas de todas las columnas:

$$f_1(s) = \sum_{j=1}^W h_j. \quad (7)$$

Un valor alto de esta caracteristica indica que la pila de piezas se encuentra cerca de la parte superior del tablero.

4.2. Huecos

Un hueco es una celda vacia que tiene al menos una celda ocupada por encima en la misma columna. La cantidad total de huecos se representa como:

$$f_2(s) = \text{numero de huecos del tablero}. \quad (8)$$

Esta caracteristica es importante porque los huecos dificultan la eliminacion de lineas y suelen empeorar la calidad del tablero.

4.3. Rugosidad

La rugosidad mide la irregularidad de la superficie del tablero. Se calcula como la suma de las diferencias absolutas entre alturas de columnas consecutivas:

$$f_3(s) = \sum_{j=1}^{W-1} |h_j - h_{j+1}|. \quad (9)$$

Un tablero con alta rugosidad dificulta la colocacion eficiente de nuevas piezas.

4.4. Altura maxima

La altura maxima corresponde a la columna mas alta del tablero:

$$f_4(s) = \max_{1 \leq j \leq W} h_j. \quad (10)$$

Esta caracteristica permite estimar que tan cerca se encuentra el agente del estado terminal.

4.5. Pozos

Un pozo es una columna o region vertical vacia rodeada por columnas de mayor altura. La cantidad de pozos se representa como:

$$f_5(s) = \text{numero de pozos del tablero}. \quad (11)$$

Los pozos pueden ser utiles si permiten colocar una pieza larga, pero en general aumentan el riesgo si no son controlados adecuadamente.

4.6. Lineas eliminadas

El numero de lineas eliminadas despues de colocar una pieza se define como:

$$f_6(s) = L(s, a). \quad (12)$$

Esta caracteristica se relaciona directamente con el objetivo principal del juego.

4.7. Vector final de caracteristicas

En conjunto, la representacion utilizada por el agente es:

$$\phi(s) = (\text{altura agregada, huecos, rugosidad, altura maxima, pozos, lineas eliminadas}). \quad (13)$$

Esta representacion permite reducir la complejidad del problema y facilita el uso de una funcion de valor lineal.

5. Funcion de Valor y Politica

El objetivo del agente es aprender una politica de decision que maximice el numero esperado de lineas eliminadas durante una partida de Tetris.

Una politica se denota por π y determina que accion debe ejecutar el agente en cada estado. En este trabajo, una accion corresponde a elegir una rotacion y una columna de colocacion para la pieza actual.

El retorno acumulado desde un instante t se define como:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \quad (14)$$

donde T es el instante en el que termina la partida, R_{t+k+1} es la recompensa obtenida y γ es el factor de descuento.

La funcion de valor bajo una politica π se define como:

$$V^\pi(s) = E_\pi(G_t \mid S_t = s). \quad (15)$$

Esta funcion estima que tan conveniente es encontrarse en el estado s si el agente continua actuando de acuerdo con la politica π .

Debido a que Tetris posee una cantidad muy grande de estados posibles, no se utiliza una tabla de valores. En su lugar, se aproxima la función de valor mediante una combinación lineal de características:

$$V(s; w) = w^T \phi(s). \quad (16)$$

En esta expresión, $\phi(s)$ es el vector de características del estado y w es el vector de pesos aprendidos. Cada peso indica la importancia de una característica dentro de la evaluación del tablero.

Para elegir una acción, el agente genera todas las colocaciones posibles de la pieza actual. Cada acción produce un tablero resultante o afterstate. Luego, el agente evalúa cada afterstate mediante la función de valor y selecciona la acción con mayor valor estimado:

$$\pi(s) = \arg \max_{a \in A(s)} V(f(s, a); w). \quad (17)$$

Aquí, $A(s)$ es el conjunto de acciones disponibles en el estado s , y $f(s, a)$ representa el tablero resultante después de ejecutar la acción a .

Por tanto, el problema de aprendizaje consiste en encontrar un vector de pesos w que permita evaluar correctamente los tableros y seleccionar mejores acciones. Esto puede expresarse como:

$$w^* = \arg \max_w J(w). \quad (18)$$

La función objetivo $J(w)$ representa el desempeño esperado del agente:

$$J(w) = E \left(\sum_{t=0}^T R_t \right). \quad (19)$$

En el proyecto se consideran dos formas de obtener los pesos w . La primera es el aprendizaje por diferencias temporales TD(0), que actualiza los pesos durante la interacción con el entorno. La segunda es el método de entropía cruzada, que busca directamente buenos vectores de pesos mediante una estrategia poblacional.

6. Aprendizaje por Diferencias Temporales

El Aprendizaje por Diferencias Temporales (Temporal Difference Learning, TD Learning) constituye una familia de algoritmos de Aprendizaje por Refuerzo cuyo objetivo es estimar funciones de valor a partir de la experiencia obtenida durante la interacción con el entorno.

A diferencia de los métodos Monte Carlo, los algoritmos TD actualizan las estimaciones antes de que el episodio haya finalizado, utilizando la información del siguiente estado como una aproximación del retorno futuro. Esta característica permite acelerar el proceso de aprendizaje y reducir la varianza de las actualizaciones.

Sea $V(s)$ la estimación de la función de valor para el estado s . El error de Diferencia Temporal se define como

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t), \quad (20)$$

donde:

- R_{t+1} es la recompensa inmediata;
- S_t es el estado actual;
- S_{t+1} es el siguiente estado;

- γ es el factor de descuento.

En el algoritmo TD(0), la función de valor se actualiza mediante

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t, \quad (21)$$

donde α representa la tasa de aprendizaje.

Debido al enorme número de estados posibles en Tetris, resulta inviable almacenar explícitamente una tabla de valores. Por esta razón, se emplea una aproximación lineal de la función de valor:

$$V(s; w) = w^T \phi(s), \quad (22)$$

donde:

- $\phi(s)$ es el vector de características;
- w es el vector de pesos del modelo.

Sustituyendo la aproximación lineal en la ecuación de actualización, se obtiene la regla de aprendizaje utilizada por el agente:

$$w \leftarrow w + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) \phi(S_t). \quad (23)$$

El agente utiliza una política ε -greedy para equilibrar exploración y explotación. Con probabilidad ε se selecciona una acción aleatoria y con probabilidad $1 - \varepsilon$ se elige la acción que maximiza la función de valor.

La principal ventaja de TD(0) radica en su capacidad para aprender de manera incremental y en línea, permitiendo actualizar continuamente la función de valor conforme el agente adquiere nueva experiencia.

7. Metodo de Entropia Cruzada

El Metodo de Entropia Cruzada (Cross-Entropy Method, CEM) es un algoritmo de optimización estocástica utilizado para resolver problemas de optimización en espacios de gran dimensionalidad.

En el contexto del presente trabajo, CEM se emplea para optimizar directamente el vector de pesos de la función de valor aproximada.

El algoritmo mantiene una distribución de probabilidad sobre los parámetros del modelo. En particular, se considera una distribución normal multivariable:

$$w \sim \mathcal{N}(\mu, \Sigma), \quad (24)$$

donde:

- μ es el vector de medias;
- Σ es la matriz de covarianzas.

En cada iteración se generan N vectores de pesos:

$$w^{(1)}, w^{(2)}, \dots, w^{(N)} \sim \mathcal{N}(\mu, \Sigma). \quad (25)$$

Cada individuo es evaluado ejecutando una partida de Tetris y calculando el retorno total:

$$J(w) = \sum_{t=0}^T R_t. \quad (26)$$

Posteriormente se selecciona un conjunto elite formado por los individuos de mayor rendimiento:

$$\mathcal{E} = \left\{ w^{(i)} : J(w^{(i)}) \geq \gamma_q \right\}, \quad (27)$$

donde γ_q representa el percentil asociado al umbral de seleccion.

A partir del conjunto elite se actualizan los parametros de la distribucion:

$$\mu' = \frac{1}{|\mathcal{E}|} \sum_{w \in \mathcal{E}} w, \quad (28)$$

$$\Sigma' = \frac{1}{|\mathcal{E}|} \sum_{w \in \mathcal{E}} (w - \mu')(w - \mu')^T. \quad (29)$$

El procedimiento se repite hasta alcanzar un numero determinado de generaciones o hasta que la distribucion converge.

La principal ventaja de CEM es que no requiere el calculo de gradientes ni una formulacion diferenciable del problema, permitiendo optimizar directamente politicas de juego mediante una estrategia poblacional.

8. Algoritmo General de Entrenamiento

El procedimiento general de entrenamiento del agente se resume en el Algoritmo 1.

Algorithm 1 Entrenamiento del Agente

```

1: Inicializar el entorno Tetris
2: Inicializar los parametros del modelo
3: for cada episodio de entrenamiento do
4:   Reiniciar el tablero
5:   Obtener el estado inicial
6:   while el episodio no haya terminado do
7:     Generar las acciones factibles
8:     Evaluar los afterstates
9:     Seleccionar una accion
10:    Ejecutar la accion
11:    Obtener recompensa y siguiente estado
12:    if se utiliza TD(0) then
13:      Actualizar los pesos mediante la ecuacion de Diferencia Temporal
14:    end if
15:  end while
16: end for
17: if se utiliza CEM then
18:   Generar poblacion de candidatos
19:   Evaluar cada individuo
20:   Seleccionar conjunto elite
21:   Actualizar la distribucion de parametros
22: end if
23: Devolver el mejor vector de pesos encontrado.
```

9. Arquitectura del Sistema

La arquitectura del sistema se organiza en módulos independientes que separan el entorno de simulación, la generación de acciones, la extracción de características, la evaluación de estados y los métodos de aprendizaje. Esta separación permite comparar TD(0) y el Método de Entropía Cruzada sobre una misma función de valor.

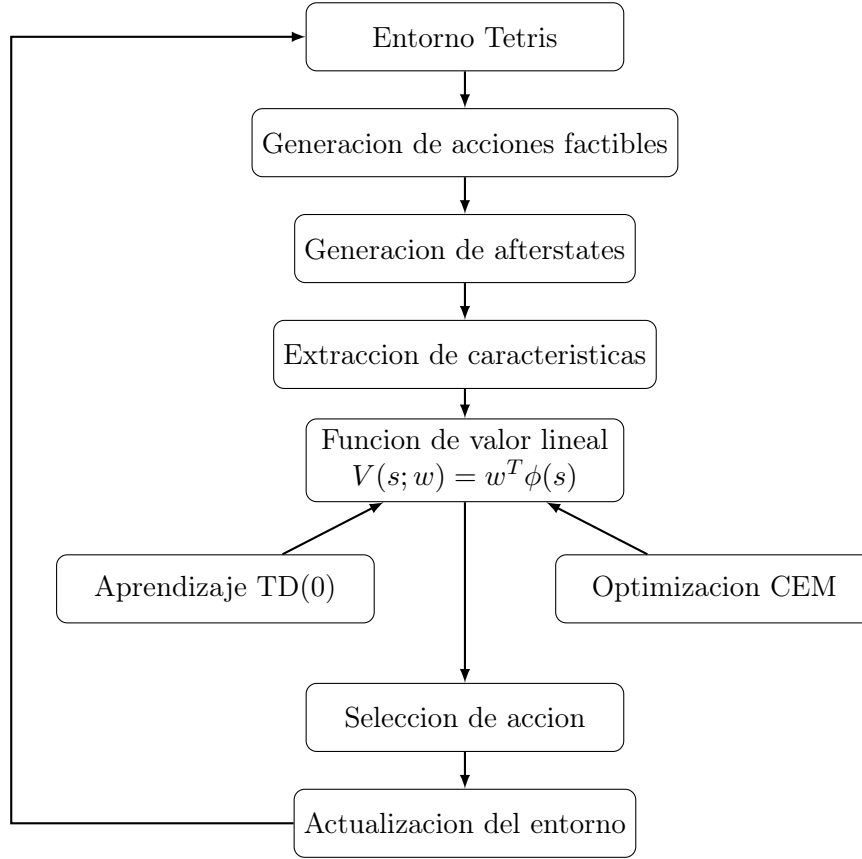


Figura 1: Arquitectura general del agente inteligente para Tetris.

El flujo inicia con el entorno de Tetris, que proporciona el tablero actual y la pieza activa. A partir de este estado se generan las acciones factibles, definidas por una rotación y una columna de colocación. Cada acción produce un tablero resultante o *afterstate*, que luego se transforma en un vector de características.

Los métodos TD(0) y CEM buscan obtener un vector de pesos adecuado para la función de valor. TD(0) actualiza los pesos de forma incremental durante la interacción con el entorno, mientras que CEM realiza una búsqueda poblacional sobre posibles vectores de pesos.

10. Diseño Experimental

Con el propósito de determinar la configuración de entrenamiento más adecuada para el agente de Aprendizaje por Refuerzo, se realizó un barrido exhaustivo de hiperparámetros mediante una estrategia de búsqueda en malla (*Grid Search*).

Se consideraron tres valores para el factor de descuento, tres valores para la tasa de aprendizaje y tres configuraciones para el decaimiento del parámetro de exploración, dando lugar a un total de

$$N_{\text{exp}} = 3 \times 3 \times 3 = 27 \quad (30)$$

configuraciones independientes de entrenamiento.
Los hiperparámetros evaluados fueron los siguientes:

- Factor de descuento:

$$\gamma \in \{0,90, 0,99, 0,999\}. \quad (31)$$

- Tasa de aprendizaje:

$$\alpha \in \{10^{-3}, 5 \times 10^{-4}, 10^{-4}\} \quad (32)$$

- Número de episodios de decaimiento de la exploración:

$$N_\epsilon \in \{1000; 1500; 1800\} \quad (33)$$

Cada configuración fue entrenada de manera independiente y posteriormente evaluada mediante el número promedio de líneas eliminadas antes de alcanzar el estado terminal del juego.

La métrica de evaluación utilizada fue

$$J = \frac{1}{N} \sum_{i=1}^N L_i, \quad (34)$$

donde L_i representa el número de líneas eliminadas en el episodio i y N corresponde al número total de episodios de evaluación.

La elección de esta métrica obedece a que el objetivo principal del agente consiste en maximizar la supervivencia y el número de líneas eliminadas durante una partida de Tetris.

11. Resultados Experimentales

Esta sección presenta los resultados obtenidos durante el barrido exhaustivo de hiperparámetros aplicado al agente de Aprendizaje por Refuerzo. El objetivo del experimento fue identificar la configuración que maximiza el número promedio de líneas eliminadas antes de alcanzar el estado terminal del juego.

En total se evaluaron veintisiete configuraciones independientes, obtenidas a partir de la combinación de tres valores para el factor de descuento γ , tres valores para la tasa de aprendizaje inicial α y tres valores para el número de episodios de decaimiento de la exploración N_ϵ .

La métrica principal de evaluación fue el promedio de líneas eliminadas por episodio:

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i, \quad (35)$$

donde L_i representa el número de líneas eliminadas en la partida i y N corresponde al número total de partidas consideradas en la evaluación.

11.1. Mejores configuraciones del barrido

La Tabla 1 muestra las diez configuraciones con mejor desempeño. Se observa que el mejor modelo corresponde a la configuración $\gamma = 0,999$, $\alpha = 0,001$ y $N_\epsilon = 1000$, la cual alcanzó un promedio de 431,21 líneas eliminadas antes del estado terminal.

El mejor resultado cuantitativo del barrido fue:

$$\gamma = 0,999, \quad \alpha = 0,001, \quad N_\epsilon = 1000, \quad \bar{L} = 431,21. \quad (36)$$

Este resultado constituye la mejor política encontrada durante la experimentación y evidencia que el agente es capaz de aprender una estrategia de juego considerablemente superior a configuraciones menos favorables.

Cuadro 1: Diez mejores configuraciones del barrido.

Ranking	γ	α	N_ϵ	Líneas promedio
1	0.999	0.0010	1000	431.21
2	0.900	0.0001	1800	408.12
3	0.990	0.0010	1000	399.07
4	0.999	0.0005	1000	386.24
5	0.990	0.0005	1800	352.79
6	0.999	0.0010	1500	352.52
7	0.990	0.0005	1000	351.37
8	0.999	0.0001	1000	351.23
9	0.990	0.0005	1500	349.42
10	0.990	0.0001	1500	343.24

Cuadro 2: Resumen del desempeño agrupado por factor de descuento.

γ	Promedio	Desviación estándar	Mínimo	Máximo
0.900	52.65	133.50	0.00	408.12
0.990	340.64	31.30	284.08	399.07
0.999	343.96	43.32	294.22	431.21

11.2. Resumen por factor de descuento

La Tabla 2 resume el desempeño agregado según el valor del factor de descuento. Los resultados muestran que los valores cercanos a la unidad producen mejores resultados promedio, lo cual es coherente con la naturaleza secuencial de Tetris: una colocación aparentemente conveniente en el corto plazo puede perjudicar la supervivencia futura del agente.

A partir de la Tabla 2, se observa que $\gamma = 0,999$ obtiene el mayor valor máximo y el mejor resultado global. Asimismo, $\gamma = 0,990$ presenta un promedio competitivo y una menor dispersión, lo que sugiere un comportamiento más estable entre distintas tasas de aprendizaje y calendarios de exploración.

11.3. Resumen por tasa de aprendizaje

La Tabla 3 presenta el desempeño agrupado por tasa de aprendizaje inicial. Aunque el mejor modelo utiliza $\alpha = 0,001$, los promedios agregados muestran que el rendimiento depende de la interacción entre α , γ y el decaimiento de exploración, no solamente de un hiperparámetro aislado.

La configuración ganadora combina una tasa de aprendizaje relativamente mayor con un factor de descuento muy cercano a uno. Esto indica que el agente logró actualizar la función de valor con suficiente velocidad sin perder la capacidad de planificar a largo plazo.

11.4. Resumen por decaimiento de exploración

La Tabla 4 muestra el desempeño agrupado por el número de episodios usados para reducir progresivamente la exploración. El mejor resultado se obtuvo con $N_\epsilon = 1000$, aunque el promedio agregado más alto corresponde a $N_\epsilon = 1800$. Esta diferencia indica que un decaimiento más

Cuadro 3: Resumen del desempeño agrupado por tasa de aprendizaje inicial.

α	Promedio	Desviación estándar	Mínimo	Máximo
0.0001	268.29	148.57	0.00	408.12
0.0005	231.09	168.98	4.94	386.24
0.0010	237.88	179.16	0.00	431.21

Cuadro 4: Resumen del desempeño agrupado por episodios de decaimiento de exploración.

N_ϵ	Promedio	Desviación estándar	Mínimo	Máximo
1000	251.68	189.56	0.00	431.21
1500	229.06	162.94	6.03	352.52
1800	256.51	143.67	11.21	408.12

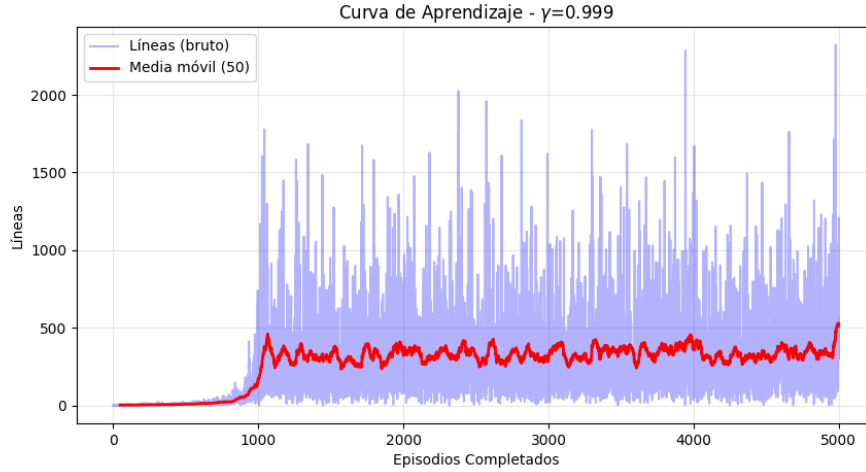


Figura 2: Curva de aprendizaje obtenida para la mejor configuración del barrido: $\gamma = 0,999$, $\alpha = 0,001$ y $N_\epsilon = 1000$.

prolongado puede mejorar la robustez general, pero la mejor configuración puntual se obtiene con una transición más rápida hacia la explotación.

11.5. Curva de aprendizaje del mejor modelo

La Figura 2 muestra la evolución del desempeño del mejor modelo durante el proceso de entrenamiento. Esta curva permite observar el comportamiento del agente conforme acumula experiencia y ajusta los pesos de la función de valor.

La curva de aprendizaje permite respaldar empíricamente que el agente no solo ejecuta una política fija, sino que modifica progresivamente su comportamiento a partir de la experiencia obtenida durante los episodios de entrenamiento.

11.6. Evolución de los pesos de la función de valor

La Figura 3 presenta la evolución temporal de los pesos asociados a la aproximación lineal de la función de valor. Cada peso representa la importancia asignada por el agente a una característica del tablero, como altura, huecos, rugosidad, pozos o líneas eliminadas.

La estabilización progresiva de los pesos constituye un indicio de que el proceso de aprendizaje converge hacia una valoración más consistente de los estados del tablero. En particular, la evolución de estos parámetros permite interpretar cómo el agente ajusta la importancia relativa de las características usadas para seleccionar acciones.

11.7. Síntesis de resultados

El barrido experimental permitió identificar una configuración capaz de eliminar en promedio 431,21 líneas por partida antes de alcanzar el estado terminal. Este resultado confirma que la combinación de Aprendizaje por Refuerzo, aproximación lineal de funciones, representación

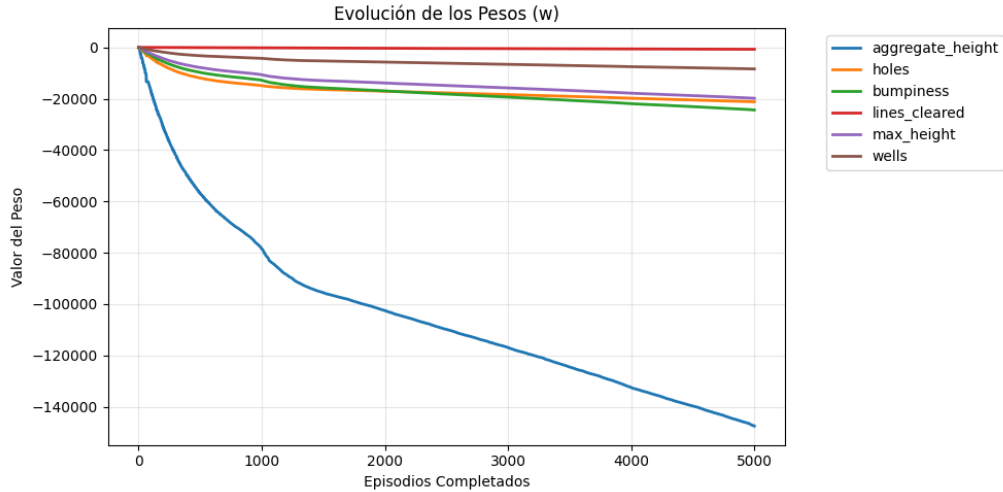


Figura 3: Evolución de los pesos de la función de valor para la mejor configuración de hiperparámetros.

mediante características y ajuste exhaustivo de hiperparámetros permite construir un agente competitivo para el entorno de Tetris considerado en este proyecto.

12. Discusión

Los resultados obtenidos permiten analizar el comportamiento del agente desde tres perspectivas: sensibilidad a los hiperparámetros, estabilidad del aprendizaje e interpretación de la política aprendida.

12.1. Sensibilidad a los hiperparámetros

El barrido exhaustivo muestra que el rendimiento del agente depende fuertemente de la combinación entre el factor de descuento, la tasa de aprendizaje y el calendario de exploración. La mejor configuración alcanzó un promedio de 431,21 líneas eliminadas, mientras que otras configuraciones obtuvieron desempeños considerablemente menores. Esta diferencia evidencia que el entrenamiento de agentes de Aprendizaje por Refuerzo no depende únicamente del algoritmo, sino también de una calibración adecuada de sus hiperparámetros.

El factor de descuento fue uno de los componentes más relevantes. Los valores $\gamma = 0,990$ y $\gamma = 0,999$ presentaron un desempeño promedio muy superior a $\gamma = 0,900$. Esto es coherente con el problema de Tetris, donde muchas decisiones correctas no producen una recompensa inmediata, sino que preparan el tablero para obtener mejores recompensas futuras. Por ejemplo, mantener una estructura baja y regular puede no eliminar líneas en el turno actual, pero aumenta la probabilidad de sobrevivir más tiempo y completar más líneas posteriormente.

La tasa de aprendizaje también mostró un efecto importante. Aunque el mejor resultado puntual se obtuvo con $\alpha = 0,001$, el promedio agregado de las tasas evaluadas indica que el desempeño no puede explicarse por este parámetro de manera aislada. Una tasa mayor puede acelerar la adaptación de los pesos, pero también aumenta el riesgo de inestabilidad si se combina con configuraciones desfavorables. Por ello, el mejor resultado surge de una interacción favorable entre $\alpha = 0,001$, $\gamma = 0,999$ y $N_\epsilon = 1000$.

12.2. Exploración y explotación

El parámetro N_ϵ controla la rapidez con la que disminuye la exploración del agente. En la mejor configuración, el valor $N_\epsilon = 1000$ permitió una transición relativamente temprana hacia la explotación de la política aprendida. Esto sugiere que, para el entorno estudiado, el agente logró adquirir información suficiente durante las primeras etapas del entrenamiento y se benefició de consolidar rápidamente las acciones de mayor valor estimado.

Sin embargo, el análisis agregado muestra que $N_\epsilon = 1800$ obtuvo el promedio más alto entre los tres valores evaluados. Esto indica que una exploración más prolongada puede mejorar la robustez general del entrenamiento, aunque no necesariamente produce la mejor configuración individual. Por tanto, existe un compromiso entre explorar durante más tiempo para cubrir mejor el espacio de estados y explotar más temprano para estabilizar la política.

12.3. Interpretación de la curva de aprendizaje

La Figura 2 permite observar la evolución del agente durante el entrenamiento. En este tipo de problemas es esperable que la curva no sea estrictamente monótona, debido a la aleatoriedad en la generación de piezas y a la exploración inducida por la política ϵ -greedy.

Aun con dicha variabilidad, la curva resulta útil para verificar que el agente experimenta una mejora progresiva y que el entrenamiento no corresponde a una ejecución aleatoria sin aprendizaje. La existencia de una tendencia de mejora y posterior estabilización respalda que la función de valor aproxima gradualmente una valoración útil de los estados del tablero.

12.4. Interpretación de la evolución de pesos

La Figura 3 permite analizar cómo cambian los parámetros de la función de valor durante el entrenamiento. En una aproximación lineal, cada peso determina la influencia de una característica sobre la evaluación del tablero. Por ello, la evolución de los pesos proporciona una forma de interpretación del comportamiento aprendido por el agente.

Una estabilización progresiva de los pesos sugiere que el agente alcanza una valoración más consistente de las características del tablero. En términos prácticos, esto significa que el agente aprende a penalizar configuraciones desfavorables, como tableros altos, huecos internos o superficies irregulares, y a favorecer configuraciones que incrementan la supervivencia y la eliminación de líneas.

12.5. Alcance del resultado obtenido

El mejor modelo logró eliminar en promedio 431,21 líneas antes del estado terminal. Este resultado es relevante porque fue obtenido mediante una aproximación lineal relativamente simple, sin redes neuronales profundas ni búsqueda exhaustiva de largo horizonte. Por tanto, el proyecto demuestra que una representación adecuada del estado mediante características puede ser suficiente para construir un agente efectivo en una versión simplificada del problema de Tetris.

No obstante, el resultado debe interpretarse dentro de las condiciones del entorno implementado. El agente decide por colocaciones completas y no modela maniobras avanzadas dependientes del tiempo real, como desplazamientos durante la caída, rotaciones tardías o técnicas especializadas. En consecuencia, el desempeño obtenido corresponde al modelo de decisión definido en el MDP del proyecto y no necesariamente a todas las variantes posibles del juego Tetris.

12.6. Limitaciones

La principal limitación del enfoque es el uso de una función de valor lineal. Aunque esta elección mejora la interpretabilidad y reduce el costo computacional, también limita la capacidad del

agente para capturar relaciones no lineales complejas entre las características del tablero. Además, el entrenamiento sigue siendo sensible a la elección de hiperparámetros, como se evidenció en el barrido experimental.

Otra limitación es que la recompensa se basa principalmente en líneas eliminadas. Aunque esta métrica está alineada con el objetivo del juego, puede no capturar completamente criterios estratégicos intermedios, como preparar el tablero para futuras piezas, reducir riesgos de top-out o mantener opciones abiertas para distintas secuencias de piezas.

12.7. Implicancias para trabajos futuros

Los resultados obtenidos abren la posibilidad de extender el sistema hacia enfoques más expresivos, como Deep Q-Learning, redes neuronales convolucionales para representar directamente el tablero o métodos híbridos que combinen heurísticas con Aprendizaje por Refuerzo. También sería posible ampliar el espacio de acciones para incluir maniobras más cercanas al juego real, así como comparar el agente entrenado contra políticas aleatorias, heurísticas clásicas y controladores basados en búsqueda.

En síntesis, el barrido experimental confirma que el agente TD con aproximación lineal es una solución viable y efectiva para el entorno propuesto. La combinación de resultados cuantitativos, curva de aprendizaje y evolución de pesos proporciona evidencia suficiente para sostener que el agente aprende una política útil y no se limita a una estrategia aleatoria o fija.

13. Conclusiones

El presente trabajo abordó el problema de la toma de decisiones en el juego Tetris mediante técnicas de Aprendizaje por Refuerzo y optimización estocástica. El entorno fue modelado formalmente como un Proceso de Decisión de Markov, permitiendo representar el problema dentro del marco teórico del Aprendizaje por Refuerzo.

Con el fin de reducir la complejidad del espacio de estados, se empleó una representación basada en características geométricas del tablero, entre las que destacan la altura agregada, el número de huecos, la rugosidad de la superficie, los pozos y la cantidad de líneas eliminadas.

La función de valor fue aproximada mediante un modelo lineal cuyos parámetros se optimizaron utilizando dos estrategias diferentes: Aprendizaje por Diferencias Temporales TD(0) y el Método de Entropía Cruzada.

Los experimentos realizados mediante un barrido exhaustivo de hiperparámetros permitieron identificar una configuración de entrenamiento capaz de eliminar en promedio

$$\bar{L} = 431,21 \tag{37}$$

líneas antes de alcanzar el estado terminal, evidenciando la capacidad del agente para aprender políticas de juego competitivas.

Los resultados obtenidos muestran que las técnicas de aproximación lineal de funciones constituyen una alternativa efectiva para abordar problemas secuenciales de gran complejidad sin necesidad de recurrir a modelos de Aprendizaje Profundo o arquitecturas neuronales de gran escala.

Finalmente, la naturaleza modular de la arquitectura desarrollada permite la incorporación de nuevos algoritmos de aprendizaje, convirtiendo al sistema propuesto en una plataforma adecuada para futuras investigaciones en Aprendizaje por Refuerzo aplicado a videojuegos y problemas de optimización secuencial.

Referencias

Bertsekas, D. P. and Tsitsiklis, J. N. (1996). *Neuro-Dynamic Programming*. Athena Scientific.

- Dellacherie, P. (2003). Tetris artificial intelligence. *France Telecom Research*.
- Gabillon, V., Ghavamzadeh, M., and Scherrer, B. (2013). Approximate dynamic programming finally performs well in the game of Tetris. In *Advances in Neural Information Processing Systems (NIPS)*, volume 26, pages 1754–1762.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Rubinstein, R. Y. (1999). The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190.
- Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.
- Szita, I. and Lőrincz, A. (2006). Learning Tetris using the noisy cross-entropy method. *Neural Computation*, 18(12):2936–2941.
- Thiery, C. and Scherrer, B. (2009). Building controllers for Tetris. *ICGA Journal*, 32(1):3–11.